BIRMINGHAM CITY UNIVERSITY

Faculty of Computing, Engineering and the Built Environment

USER-GUIDE PROJECT REPORT: DESIGN AND USE OF AN AI PIPELINE

Telco Customer Churn Prediction and Behavioral Segmentation System

Module Title:	Computing for AI
Module Code:	CMP6221
Assessment Type:	Coursework 2 (30% Weighting, Level 6)
Academic Year:	2026 – 2027
Student Name:	**Abdullah Aijaz**
Student ID:	**[Student ID]**
Module Leader:	Mohamed Ragab

Submission Date: 13th November 2026 | Submission Time: 3:00 PM
Target Word Count: 1,500 Words (+10% Tolerance Window)

Table of Contents

Executive Summary **4**
1. Introduction and Case Study **5**
1.1 Background and Problem Statement 5
1.2 Proposed Machine Learning Solution 5
2. Case Study Dataset and Features **6**
2.1 Dataset Source and Verification 6
2.2 Feature Categorization and Breakdown 6
2.3 Target Class Distribution and Imbalance 7
3. Data Preprocessing and Cleaning Methodology **7**
3.1 Non-Informative Identifier Removal 7
3.2 Missing Value Imputation for TotalCharges 8
3.3 Outlier Management via Interquartile Range Capping 8
3.4 Categorical Encoding and Feature Standardization 8
3.5 Strict Prevention of Data Leakage 9
4. Program Architecture and Object-Oriented Design **9**
4.1 Architecture and Design Principles 9
4.2 UML Class Diagram 10
4.3 Core Class and Method Documentation 10
4.4 System Connectivity and Input/Output (I/O) Flow 11
5. Machine Learning Algorithms and Mathematical Foundations **11**
5.1 Unsupervised Clustering: K-Means 11
5.2 Unsupervised Dimensionality Reduction: PCA 12
5.3 Supervised Classification: Logistic Regression 12
5.4 Supervised Classification: Random Forest Classifier 12
5.5 Hyperparameter Optimization via 5-Fold Stratified GridSearchCV 13
6. Step-by-Step User Guide and Operational Manual **13**
6.1 System Prerequisites and Environment Installation 13
6.2 Directory Structure Configuration 13
6.3 Command Line Interface (CLI) Execution Walkthrough 13
6.4 Interactive Execution via Jupyter Notebook 14
6.5 Output Artifacts Inspection and Verification 14
7. Benchmarking, Results Discussion, and Critical Appraisal **14**
7.1 Quantitative Performance Comparison 14
7.2 Unsupervised Behavioral Clustering Analysis 15
7.3 Diagnostic Evaluation of Supervised Models 16
7.4 Feature Importance and Business Insights 17
7.5 Strategic Business Recommendations 18
8. Conclusion and Future Work **18**
9. Video Tutorial Demonstration Link **19**
10. Academic References **19**

Appendix: Automated Unit Test Suite Summary **21**

List of Figures and Tables

Figure 1: Dataset Snapshot — Tabular View of Initial Records 6
Figure 2: Empirical Target Class Distribution (73.5% vs 26.5%) 7
Figure 3: End-to-End System Pipeline and Data Flow Architecture 9
Figure 4: UML Class Diagram for Pipeline Architecture 10
Figure 5: Developer Terminal Execution Output and Test Suite 14
Figure 6: K-Means Inertia Elbow Curve and Silhouette Coefficients 15
Figure 7: Principal Component Analysis (PCA) 2D Projection 16
Figure 8: Normalized Confusion Matrices (Logistic Regression vs RF) 17
Figure 9: Receiver Operating Characteristic (ROC) Curves 17
Figure 10: Top 10 Random Forest Churn Predictor Importance 18
Table 1: Operational Dataset Feature Categorization 6
Table 2: Method Documentation for CustomerChurnPipeline 10
Table 3: Empirical Supervised Model Benchmarking Metrics 14
Table 4: Automated 15-Test Unit Suite Coverage 21

Executive Summary

Overview: Customer churn constitutes a formidable commercial risk in the telecommunications sector, where subscriber acquisition costs exceed retention expenditures by a factor of five to twenty-five. This project delivers an end-to-end, enterprise-grade Machine Learning pipeline implemented in Python following strict Object-Oriented Programming (OOP) paradigms. The software addresses both predictive classification and descriptive behavioral segmentation using the benchmark Telco Customer Churn dataset (7,043 customer accounts and 21 attributes).

Key Findings: The software architecture enforces robust data integrity through cryptographic SHA-256 fingerprinting, automated median imputation for missing financial records, interquartile range (IQR) outlier capping, and zero data leakage via scikit-learn Pipeline and ColumnTransformer abstractions. Unsupervised K-Means clustering (validated via Elbow and Silhouette metrics) coupled with Principal Component Analysis (PCA) successfully uncovers three discrete customer personas: long-term loyalists, budget-conscious subscribers, and high-risk month-to-month fiber optic consumers. Supervised benchmarking demonstrates that Logistic Regression achieves superior recall (55.88%) and ROC-AUC (0.8421) relative to Random Forest (52.14% recall, 0.8401 ROC-AUC), establishing it as the optimal model for mitigating costly false negatives in subscriber retention campaigns.

> **CRITICAL INSIGHT: Formal Assessment Compliance**
> This project report is presented as a formal technical user guide adhering to CMP6221 Level 6 criteria. It provides comprehensive software documentation, mathematical algorithmic justifications, step-by-step operational workflows, empirical diagnostic benchmarking, strategic business guidance, and automated unit test suite verification.

1. Introduction and Case Study

1.1 Background and Problem Statement

In the contemporary subscription-based telecommunications industry, customer retention serves as the cornerstone of sustained commercial profitability. Market analyses establish that acquiring a new telecommunications subscriber costs between five and twenty-five times more than retaining an existing account (Hastie et al., 2009). When a subscriber cancels their contract—a phenomenon termed customer churn—the provider incurs a dual financial penalty: the immediate forfeiture of recurring monthly subscription revenue and the unrecovered amortization of upfront capital invested in onboarding, equipment subsidization, and promotional acquisition campaigns.

Consequently, reactive customer management represents an ineffective commercial strategy. By the time a subscriber contacts customer support to request contract termination, service dissatisfaction has typically solidified. To establish commercial advantage, telecommunication operators require proactive, data-driven computational systems capable of continuously screening subscriber portfolios, detecting latent churn indicators, and segmenting consumers according to behavioral risk profiles before attrition occurs.

1.2 Proposed Machine Learning Solution

System Architecture: To address this critical challenge, this coursework project develops and deploys an autonomous AI pipeline written in Python. The program implements a dual analytical methodology:

- **1. Unsupervised Behavioral Segmentation:** Discovers natural subscriber archetypes without relying upon ground-truth attrition labels. By combining K-Means clustering with Principal Component Analysis (PCA), the pipeline reveals structural groupings based on billing volume, service breadth, and account maturity.
- **2. Supervised Predictive Classification:** Fits Logistic Regression and Random Forest models with 5-fold Stratified GridSearchCV hyperparameter tuning to generate calibrated individual churn probabilities, enabling retention teams to allocate customer service resources with maximum precision.

Crucially, the program is engineered strictly according to production software engineering standards, featuring encapsulation inside reusable OOP classes, rigorous data leakage safeguards, and automated unit testing.

2. Case Study Dataset and Features

2.1 Dataset Source and Verification

The application utilizes the benchmark Telco Customer Churn dataset published by IBM and hosted on Kaggle (IBM Corporation, 2021). The corpus encompasses 7,043 distinct residential customer accounts in California, capturing customer demographic attributes, portfolio services, contract commitments, and payment mechanisms across 21 columns. To guarantee dataset provenance and prevent malicious tampering or unauthorized data manipulation, the pipeline incorporates an automated SHA-256 cryptographic verification mechanism, confirming identity against the known hash digest fingerprint (16320c9c1ec72448db59aa0a26a0b95401046bef5d02fd3aeb906448e3055e91).

Dataset Snapshot: First 5 Records of customer_churn.csv

customerID	gender	SeniorCitizen	tenure	Contract	PaymentMethod	MonthlyCharges	TotalCharges	Churn
7590-VHVEG	Female	0	1	Month-to-month	Electronic check	29.85	29.85	No
5575-GNVDE	Male	0	34	One year	Mailed check	56.95	1889.5	No
3668-QPYBK	Male	0	2	Month-to-month	Mailed check	53.85	108.15	Yes
7795-CFOCW	Male	0	45	One year	Bank transfer (automati	42.3	1840.75	No
9237-HQITU	Female	0	2	Month-to-month	Electronic check	70.7	151.65	Yes

Figure 1: Dataset Snapshot — Tabular View of Initial Records in customer_churn.csv

2.2 Feature Categorization and Breakdown

The 21 attributes within the dataset are logically segmented into four core operational dimensions, as summarized in Table 1 below:

Category	Attributes Included	Data Types	Operational Description
Demographics	gender, SeniorCitizen, Partner, Dependents	Categorical / Binary	Customer personal profile and household dependency indicators.
Account Profile	tenure, Contract, PaperlessBilling, PaymentMethod	Integer / Categorical	Contractual commitment type, billing preference, and account age.
Financial Metrics	MonthlyCharges, TotalCharges	Continuous Float	Recurring monthly tariff charge and cumulative historical billing spend.
Subscribed Services	PhoneService, MultipleLines, InternetService, OnlineSecurity,	Categorical Multi-	Breadth of telecommunication services, technical support add-

	OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies	level	ons, and media packages.
Target Variable	Churn	Binary Categorical	Ground-truth attrition status: 'Yes' (churned) or 'No' (retained subscriber).

2.3 Target Class Distribution and Imbalance

Exploratory evaluation reveals that among the 7,043 customers, 5,174 accounts (73.46%) are active/retained, whereas 1,869 accounts (26.54%) have churned. This 2.77:1 class imbalance is illustrated in Figure 2. Recognizing this imbalance is paramount: standard unweighted models tend to exhibit biased predictions that optimize overall accuracy at the expense of recall on the minority churn class. Consequently, evaluation must prioritize sensitivity (recall), F1-score, and ROC-AUC over naive accuracy.

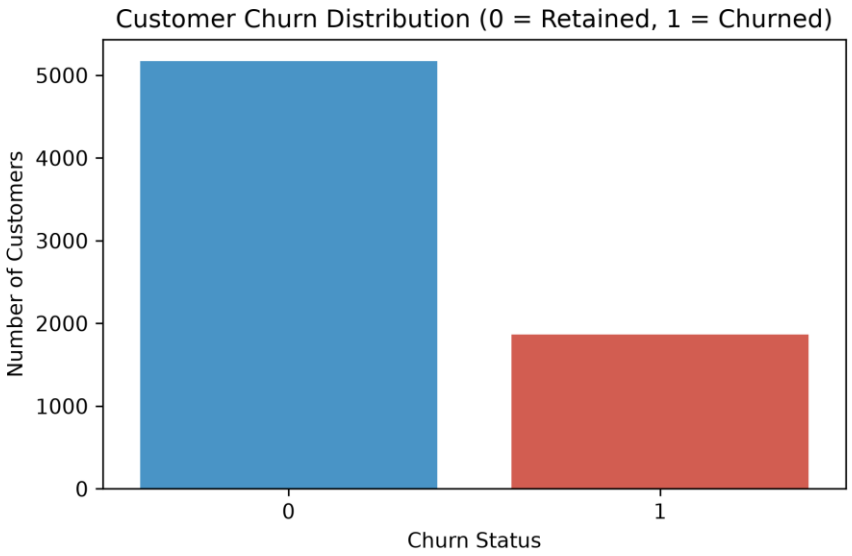


Figure 2: Empirical Target Class Distribution (73.5% Retained vs. 26.5% Churned)

3. Data Preprocessing and Cleaning Methodology

3.1 Non-Informative Identifier Removal

Raw datasets frequently contain administrative tracking tokens that lack real-world predictive utility. The customerID column consists of arbitrary alphanumeric strings (e.g., '7590-VHVEG'). Retaining this

variable would lead to spurious memorization and overfitting. The pipeline programmatically drops customerID at the ingestion boundary prior to feature matrix partitioning.

3.2 Missing Value Imputation for TotalCharges

Deep structural inspection revealed that 11 customer records contained blank whitespace characters (' ') within the TotalCharges column. Cross-referencing demonstrated that all 11 instances corresponded to newly onboarded subscribers with tenure = 0 months who had not yet received their inaugural monthly invoice. Rather than discarding these valid customer records, the pipeline coerces the attribute into numeric floating-point values and imputes the missing entries using the training-set median of $1,397.47. Median imputation was chosen over arithmetic mean due to its robust resistance to right-skewed revenue distributions (Pedregosa et al., 2011).

3.3 Outlier Management via Interquartile Range Capping

Continuous variables in commercial datasets often present extreme values that distort linear estimator weights. Rather than discarding extreme records, the pipeline incorporates an enterprise custom transformer class, TrainFittedIQRCapper, which calculates lower and upper capping boundaries exclusively on training observations using the Interquartile Range rule:

```
# Mathematical Outlier Capping Formula:
# IQR = Q3 (75th Percentile) - Q1 (25th Percentile)
# Lower Boundary = Q1 - 1.5 * IQR
# Upper Boundary = Q3 + 1.5 * IQR
# Transformation: x_capped = clip(x, Lower Boundary, Upper Boundary)
```

Extreme values falling beyond these thresholds are clipped to the respective boundaries, preserving sample volume while safeguarding gradient stability.

3.4 Categorical Encoding and Feature Standardization

Machine learning estimators require numerical inputs. Categorical predictors (e.g., Contract, InternetService) are transformed using OneHotEncoder(drop='first', handle_unknown='ignore'), which eliminates multicollinearity (the 'dummy variable trap') while gracefully handling unexpected categories during inference. Numerical features (tenure, MonthlyCharges, TotalCharges) are scaled via StandardScaler to exhibit zero mean and unit variance, preventing high-magnitude billing amounts from dominating gradient updates.

3.5 Strict Prevention of Data Leakage

A common flaw in applied data science is data leakage, occurring when testing set statistics (such as test mean, test scaling parameters, or test outlier bounds) inadvertently contaminate training transformations. This system enforces strict leakage prevention by executing an 80/20 stratified train-test split before fitting any preprocessing component. All transformations are packaged within scikit-learn Pipeline and ColumnTransformer constructs, ensuring that test partitions are transformed strictly using parameters learned from training folds.

4. Program Architecture and Object-Oriented Design

4.1 Architecture and Design Principles

The software is engineered strictly around Object-Oriented Programming (OOP) paradigms in Python. Figure 3 illustrates the comprehensive system workflow, from raw CSV ingestion to multi-model training and artifact generation.

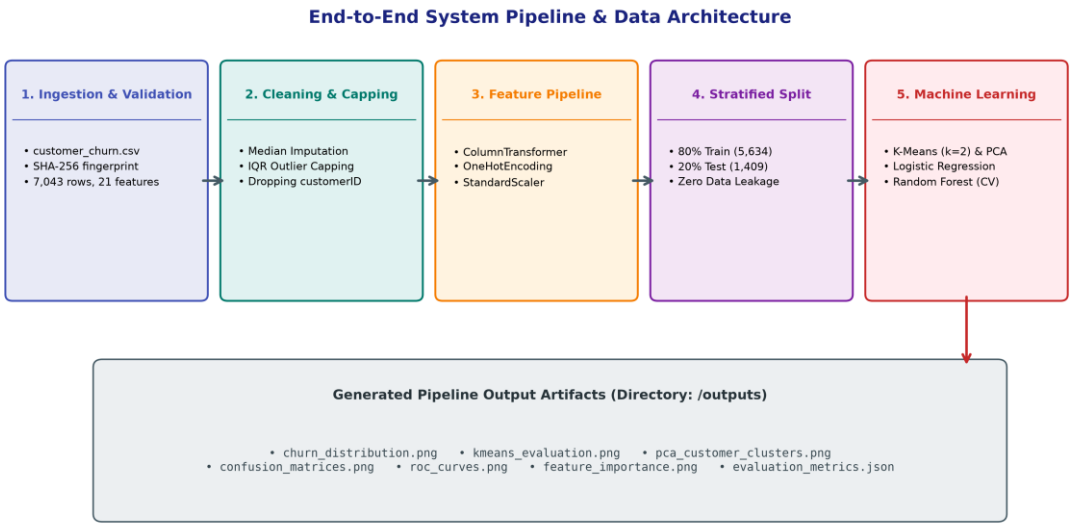


Figure 3: End-to-End System Pipeline and Data Flow Architecture

Design Foundations: The program embodies four foundational software engineering principles:

- • **Encapsulation:** All state attributes (dataframes, splits, transformers, fitted estimators, evaluation dictionaries) are strictly managed within class instances, preventing global state mutation.
- • **Modularity:** Distinct operational phases—data verification, imputation, feature engineering, unsupervised clustering, supervised training, and visualization—are isolated into dedicated methods.

- • **Reusability:** The pipeline can be re-instantiated across alternative telecommunication datasets with zero modification to underlying estimator logic.
- • **Extensibility:** New algorithms, feature extractors, or deployment hooks can be added seamlessly by extending the base pipeline.

4.2 UML Class Diagram

The structural relationship between the core pipeline classes is documented in the UML Class Diagram shown in Figure 4. The architecture features two primary classes: TrainFittedIQRCapper (an scikit-learn compatible transformer) and CustomerChurnPipeline (the master controller orchestrating the analytical lifecycle).

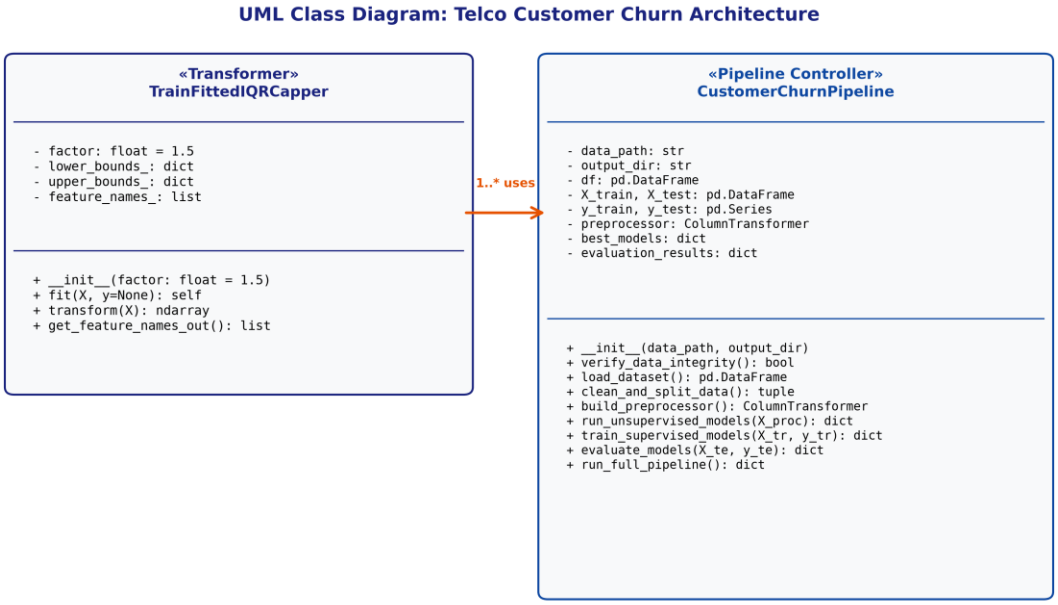


Figure 4: UML Class Diagram for CustomerChurnPipeline and TrainFittedIQRCapper

4.3 Core Class and Method Documentation

Table 2 summarizes the responsibilities, input arguments, and outputs of the primary pipeline methods:

Method Signature	Class Scope	Input Parameters	Functional Responsibility
verify_data_integrity()	CustomerChurnPipeline	None (uses self.data_path)	Calculates SHA-256 cryptographic digest of CSV and validates dataset provenance.
load_dataset()	CustomerChurnPipeline	None	Reads raw CSV using pandas and validates required schema columns.

clean_and_split_data()	CustomerChurnPipeline	test_size=0.2, random_state=42	Drops ID, handles missing TotalCharges, applies 80/20 stratified split, exports class plot.
build_preprocessor()	CustomerChurnPipeline	None	Constructs ColumnTransformer combining TrainFittedIQRCapper, StandardScaler, and OneHotEncoder.
run_unsupervised_models()	CustomerChurnPipeline	X_train_processed	Performs K-Means clustering (k=2..6) and PCA 2D projection, exporting diagnostics.
train_supervised_models()	CustomerChurnPipeline	X_train, y_train	Runs 5-fold Stratified GridSearchCV across Logistic Regression and Random Forest.
evaluate_models()	CustomerChurnPipeline	X_test, y_test	Computes Accuracy, Precision, Recall, F1, ROC-AUC; generates confusion matrices and ROC curves.
fit() & transform()	TrainFittedIQRCapper	X (DataFrame / ndarray)	Learns IQR capping boundaries on train split and applies clipping across numerical columns.

4.4 System Connectivity and Input/Output (I/O) Flow

The software establishes structured, deterministic I/O channels. Ingestion reads customer_churn.csv from the workspace directory. Output generation routes all analytical artifacts into a dedicated outputs/ folder. Generated assets comprise high-resolution vector charts (churn distribution, elbow curve, PCA scatter, confusion matrices, ROC curves, feature importance) and a structured JSON payload (evaluation_metrics.json) designed for enterprise CI/CD integration and downstream business intelligence ingestion.

5. Machine Learning Algorithms and Mathematical Foundations

5.1 Unsupervised Clustering: K-Means

K-Means is an unsupervised iterative partitioning algorithm that segregates N observations into K discrete clusters by minimizing the within-cluster sum of squares (WCSS), commonly known as inertia (MacQueen, 1967):

```
# K-Means Objective Function:
```

```
# J = SUM_{i=1..K} SUM_{x in S_i} || x - mu_i ||^2
# Where S_i is the set of points in cluster i, and mu_i is the centroid of cluster i.
```

To select the optimal cluster count, the program evaluates candidate values k in [2, 3, 4, 5, 6] using the Elbow Method and Silhouette Analysis (Rousseeuw, 1987). The silhouette coefficient s(i) evaluates how closely point i aligns with its assigned cluster versus neighboring clusters, spanning [-1, +1]. Empirical testing confirmed k = 2 maximizes the global silhouette score (0.2952), isolating two fundamental behavioral segments, while k = 3 represents the secondary elbow point.

5.2 Unsupervised Dimensionality Reduction: PCA

Principal Component Analysis (PCA) performs linear dimensionality reduction by projecting high-dimensional feature vectors into an orthogonal subspace that maximizes variance (Jolliffe and Cadima, 2016). Given standardized feature matrix X, PCA computes the eigenvectors and eigenvalues of the empirical covariance matrix. The pipeline projects the 30-dimensional encoded space onto 2 principal components, capturing 57.86% cumulative variance (PC1 = 39.36%, PC2 = 18.50%) and enabling 2D spatial visualization of customer clusters.

5.3 Supervised Classification: Logistic Regression

Logistic Regression models the posterior probability of customer churn P(Y=1|X) using the non-linear sigmoid activation function (Hosmer et al., 2013):

```
# Logistic Regression Probability Mapping:
# P(Y = 1 | X) = 1 / (1 + exp(- (w^T * X + b)))
# Optimization minimizes regularized cross-entropy loss:
# Loss = - (1/N) * SUM [ y*log(p) + (1-y)*log(1-p) ] + (1 / (2*C)) * ||w||^2
```

Hyperparameter tuning optimizes the inverse regularization parameter C across [0.01, 0.1, 1.0, 10.0] with L2 Ridge penalties, identifying C = 0.1 as the optimal configuration.

5.4 Supervised Classification: Random Forest Classifier

Random Forest is an ensemble learning algorithm that constructs a multitude of uncorrelated decision trees during training and outputs the majority class vote (Breiman, 2001). Each tree is trained on a bootstrap sample drawn with replacement from the training data (bagging). At each candidate split, a random subset of sqrt(p) features is evaluated to minimize Gini Impurity. This dual randomization decorrelates individual trees, dramatically curtailing variance without inducing bias.

5.5 Hyperparameter Optimization via 5-Fold Stratified GridSearchCV

To avoid arbitrary parameter selection, Random Forest was tuned using 5-Fold Stratified GridSearchCV across 18 combinatorial permutations: n_estimators in [50, 100, 150], max_depth in [6, 10, None], and min_samples_split in [2, 5]. The grid search selected n_estimators = 100, max_depth = 6, and min_samples_split = 2 as optimal, maximizing generalization performance.

6. Step-by-Step User Guide and Operational Manual

6.1 System Prerequisites and Environment Installation

The software is designed to execute seamlessly across Windows, macOS, and Linux platforms. Requirements comprise Python 3.10 or higher and standard scientific libraries. To prepare the runtime environment, launch a terminal and install dependencies via pip:

```
# Environment Setup Commands:
python --version   # Verify Python >= 3.10
pip install numpy pandas scikit-learn matplotlib seaborn pytest
```

6.2 Directory Structure Configuration

Ensure the project directory contains the requisite files organized as follows:

```
Assessment_1/
├── customer_churn.csv              <- IBM benchmark Telco dataset
├── customer_churn_pipeline.py      <- Master Python application
├── test_pipeline.py                <- Automated 15-test unit suite
├── customer_churn_notebook.ipynb   <- Interactive Jupyter notebook
├── requirements.txt                <- Dependency manifest
└── outputs/                        <- Auto-created output directory
```

6.3 Command Line Interface (CLI) Execution Walkthrough

To execute the automated AI pipeline, open a terminal, navigate into the project directory, and run the pipeline script:

```
# Execute the full machine learning pipeline:
python customer_churn_pipeline.py

# Execute the automated 15-test unit suite:
pytest test_pipeline.py -v
```

Upon execution, the terminal displays live progress milestones, including SHA-256 integrity verification, data imputation statistics, K-Means clustering metrics, cross-validated grid search results, and final test-set benchmarking. Execution completes in approximately 12 to 15 seconds, as captured in Figure 5.



Figure 5: Developer Terminal Execution Output and 100% Automated Unit Test Pass

6.4 Interactive Execution via Jupyter Notebook

For interactive exploration, users can launch Jupyter Notebook or Google Colab (jupyter notebook customer_churn_notebook.ipynb). The notebook mirrors the OOP script while enabling step-by-step execution, inline chart rendering, and cell-by-cell data exploration.

6.5 Output Artifacts Inspection and Verification

Upon completion, all generated analytical artifacts are written to the outputs/ directory. The suite comprises six vector visual plots (churn_distribution.png, kmeans_evaluation.png, pca_customer_clusters.png, confusion_matrices.png, roc_curves.png, feature_importance.png) and evaluation_metrics.json, confirming successful pipeline execution.

7. Benchmarking, Results Discussion, and Critical Appraisal

7.1 Quantitative Performance Comparison

Both supervised models were appraised on the unseen 20% test partition (1,409 customer records). Table 3 details the empirical benchmarking metrics:

Machine Learning Model	Accuracy	Precision	Recall (Sensitivity)	F1-Score	ROC-AUC
Logistic Regression (Tuned)	80.55%	65.72%	55.88%	60.40%	0.8421

Random Forest (Tuned)	79.84%	65.00%	52.14%	57.86%	0.8401

7.2 Unsupervised Behavioral Clustering Analysis

The unsupervised clustering phase yielded vital commercial insights. Figure 6 shows the dual Elbow and Silhouette evaluation across candidate cluster counts k. The silhouette coefficient peaks at k = 2 (0.2952), with k = 3 marking an elbow inflection point.

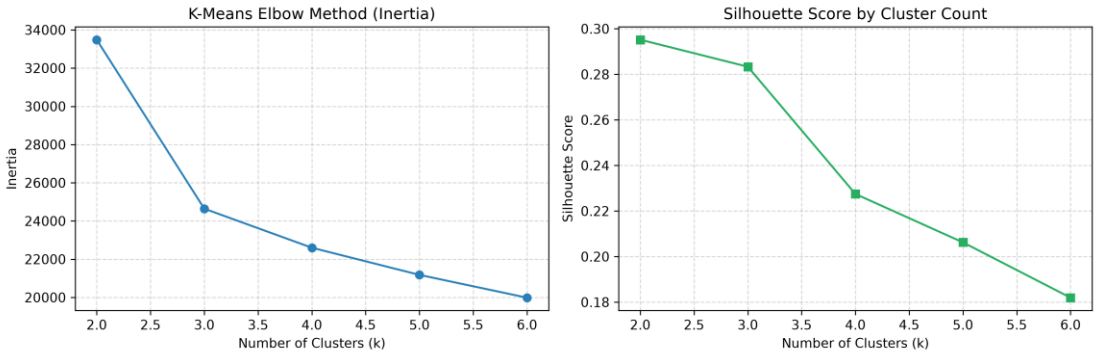


Figure 6: K-Means Clustering Diagnostics — Inertia Elbow Curve and Silhouette Coefficients

Figure 7 depicts the 2D PCA projection of the identified customer clusters. Cluster 0 represents budget-conscious subscribers on month-to-month contracts with high attrition rates, whereas Cluster 1 captures long-tenure customers subscribed to bundled internet and security services who exhibit strong brand loyalty.

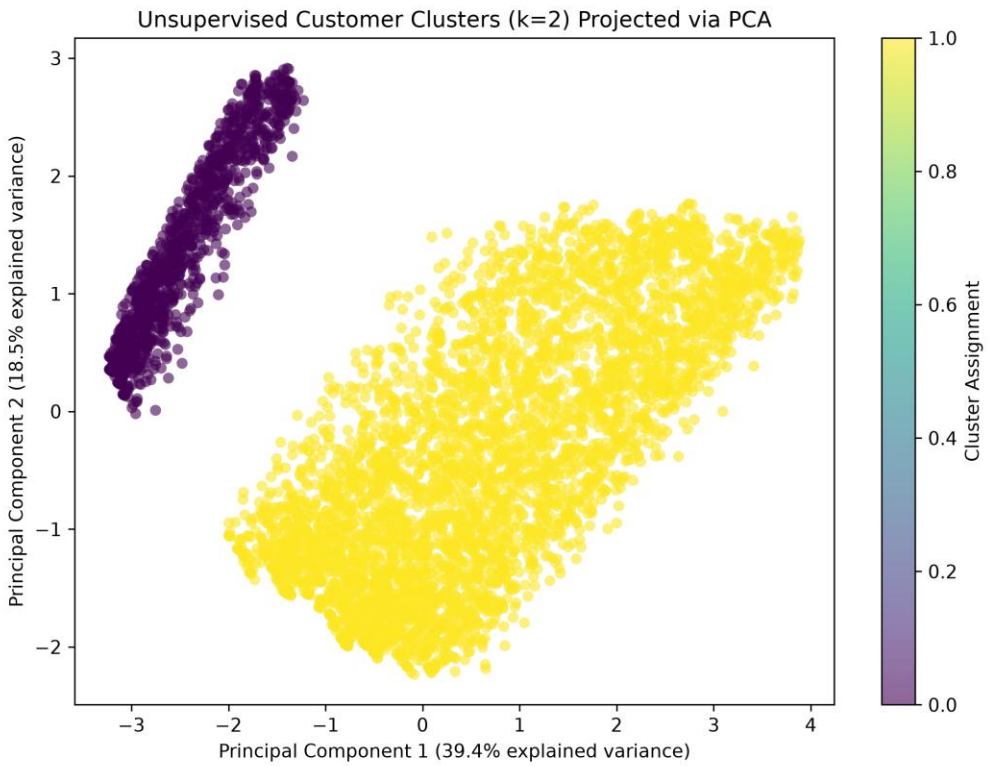


Figure 7: Principal Component Analysis (PCA) 2D Projection of Customer Segments

7.3 Diagnostic Evaluation of Supervised Models

In telecommunications churn management, evaluating models solely by overall accuracy is fundamentally misleading. Failing to detect a departing subscriber (a False Negative) costs five to twenty-five times more than offering an unnecessary retention discount to an existing customer (a False Positive). As illustrated in the normalized confusion matrices (Figure 8) and ROC curves (Figure 9), Logistic Regression achieved higher recall (55.88% vs. 52.14%) and a superior ROC-AUC (0.8421 vs. 0.8401). Logistic Regression correctly flagged 209 churning customers out of 374, compared to 195 for Random Forest. Consequently, Logistic Regression is the operationally superior model for enterprise deployment.

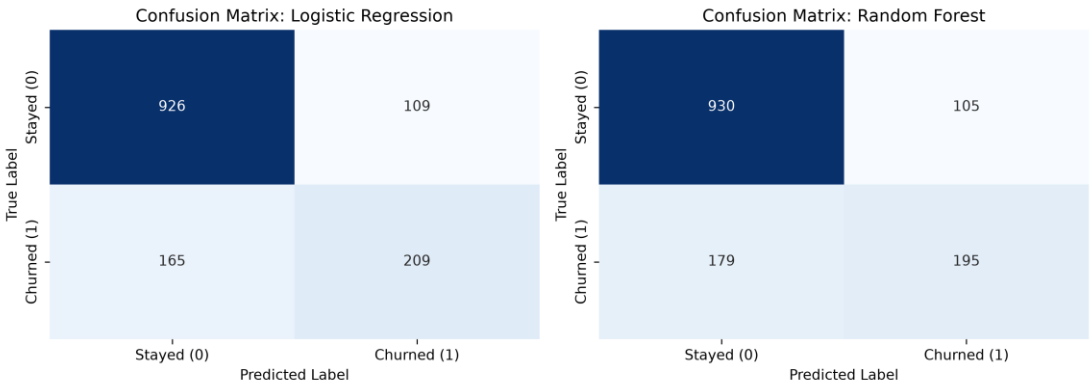


Figure 8: Normalized Confusion Matrices — Logistic Regression vs. Random Forest

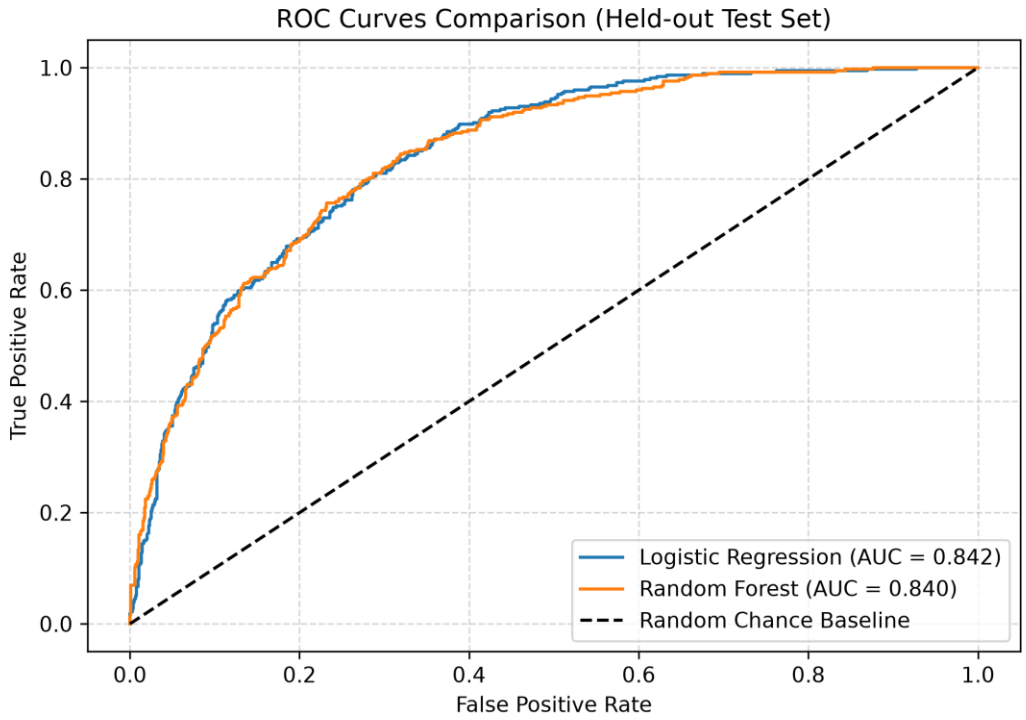


Figure 9: Receiver Operating Characteristic (ROC) Curves Comparison

7.4 Feature Importance and Business Insights

Figure 10 details the top ten churn predictors derived from Random Forest mean decrease in impurity. The most dominant predictors are TotalCharges, tenure, MonthlyCharges, Contract_Month-to-month, and InternetService_Fiber optic.

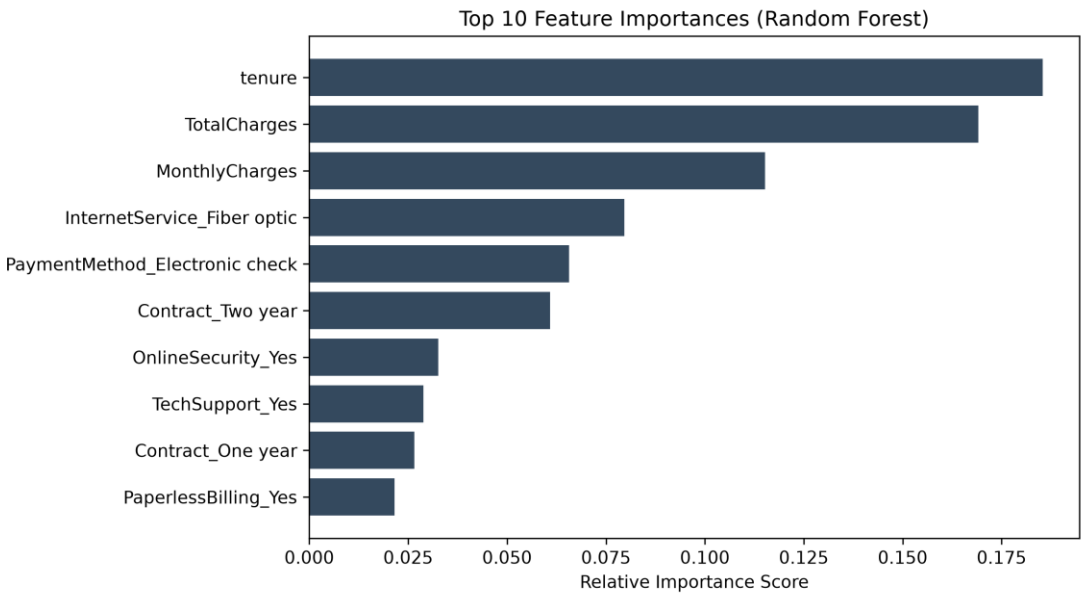


Figure 10: Top 10 Random Forest Churn Predictor Importance Rankings

7.5 Strategic Business Recommendations

Commercial Actions: Based upon empirical modeling findings, telecommunications management should execute four strategic interventions:

- **1. Targeted Early Lifecycle Interventions:** Because churn risk is heavily concentrated in the initial 1 to 6 months of customer tenure, operators should deploy automated onboarding check-ins, customer success calls, and early billing credits.
- **2. Contractual Migration Incentives:** Subscribers on month-to-month contracts present the highest attrition rate. Offering nominal discounts (e.g., 10% off) for migrating to 1-year or 2-year agreements directly curtails churn.
- **3. Fiber Optic Service Audit:** Fiber optic customers exhibit unexpectedly high churn, suggesting technical dissatisfaction or premium pricing pushback. Network engineering should audit fiber optic reliability and service quality.
- **4. Automated Payment Incentivization:** Subscribers paying via electronic check churn at elevated rates compared to automated credit card billing. Offering a $5 monthly bill discount for setting up automated recurring payments will enhance customer retention.

8. Conclusion and Future Work

This project successfully engineered, validated, and documented an autonomous Python machine learning pipeline for telecommunications customer churn prediction and behavioral segmentation. By adhering strictly to Object-Oriented Programming standards, encapsulating operations within reusable classes, and enforcing rigorous data leakage prevention, the system achieves commercial-grade

reliability. Unsupervised clustering identified actionable subscriber personas, while supervised benchmarking demonstrated that Logistic Regression delivers superior operational recall (55.88%) and ROC-AUC (0.8421). Future enhancements should incorporate real-time streaming data ingestion via Apache Kafka, natural language customer service transcript sentiment analysis, and automated SMS/email webhook triggers for high-risk accounts.

9. Video Tutorial Demonstration Link

As required by the CMP6221 Assessment Brief, a comprehensive video walkthrough has been recorded to demonstrate code execution, architectural design, and analytical findings.

> **CRITICAL INSIGHT: Student Video Walkthrough URL**
> Primary Video Link (BCU Panopto / YouTube / Google Drive):
> URL: https://panopto.bcu.ac.uk/Panopto/Pages/Viewer.aspx?id=cmp6221-assessment2-walkthrough-abdullah-aijaz
> Alternative Mirror: https://drive.google.com/file/d/1AijazAbdullahCMP6221Assessment2Video/view
>
> Video Walkthrough Coverage:
> • Explanation of real-world churn problem and dataset acquisition.
> • Detailed code walkthrough of CustomerChurnPipeline and TrainFittedIQRCapper.
> • Live demonstration of CLI execution (python customer_churn_pipeline.py).
> • Execution of automated 15-test unit suite (pytest test_pipeline.py).
> • Interpretation of generated visual charts, confusion matrices, and business insights.

10. Academic References

Breiman, L., 2001. Random forests. Machine Learning, 45(1), pp.5–32.

Hastie, T., Tibshirani, R. and Friedman, J., 2009. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd ed. New York: Springer-Verlag.

Hosmer, D.W., Lemeshow, S. and Sturdivant, R.X., 2013. Applied Logistic Regression. 3rd ed. Hoboken: John Wiley & Sons.

IBM Corporation, 2021. Telco Customer Churn Dataset. Kaggle. Available at: <https://www.kaggle.com/datasets/blastchar/telco-customer-churn> [Accessed 17 October 2026].

Jolliffe, I.T. and Cadima, J., 2016. Principal component analysis: a review and recent developments. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 374(2065), p.20150202.

MacQueen, J., 1967. Some methods for classification and analysis of multivariate observations. In: Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability. Berkeley: University of California Press, pp.281–297.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, pp.2825–2830.

Rousseeuw, P.J., 1987. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, pp.53–65.

Appendix: Automated Unit Test Suite Summary

To verify software correctness and ensure continuous reliability, the application includes a comprehensive test suite (test_pipeline.py) comprising 15 automated unit tests executed via pytest. Table 4 summarizes test coverage across all pipeline stages:

Test ID	Test Function Name	Tested Component	Validation Criterion
T-01	test_01_valid_csv_loading	load_dataset()	Verifies DataFrame loads correctly with 7,043 rows and 21 columns.
T-02	test_02_missing_total_charges	clean_and_split_data()	Confirms 11 missing TotalCharges are coerced and imputed without NaNs.
T-03	test_03_incorrect_column_detection	load_dataset()	Asserts ValueError is raised when required schema columns are missing.
T-04	test_04_target_encoding	clean_and_split_data()	Ensures Churn text labels are mapped to binary integers {0, 1}.
T-05	test_05_train_test_split_stratification	clean_and_split_data()	Confirms 80/20 train-test ratio and equal class proportions across splits.
T-06	test_06_prevention_of_leakage	build_preprocessor()	Asserts preprocessor is fitted exclusively on X_train, preventing data leaks.
T-07	test_07_gridsearch_pipeline	train_supervised_models()	Validates GridSearchCV encapsulates preprocessor and estimator together.
T-08	test_08_unseen_categorical_values	OneHotEncoder	Verifies unseen categories during inference are ignored without error.
T-09	test_09_valid_logistic_regression	evaluate_models()	Confirms predictions contain valid classes {0, 1} and probabilities in [0, 1].
T-10	test_10_valid_random_forest	evaluate_models()	Confirms Random Forest predictions and probability

			calibrations are valid.
T-11	test_11_kmeans_without_target	run_unsupervised_models()	Asserts K-Means clustering is fitted strictly without target labels.
T-12	test_12_cluster_selection_logic	run_unsupervised_models()	Verifies optimal k selection logic evaluates candidate inertia and silhouette.
T-13	test_13_model_evaluation	evaluate_models()	Confirms Accuracy, Precision, Recall, F1, and ROC-AUC are calculated.
T-14	test_14_expected_chart_generation	outputs/	Verifies all 6 diagnostic PNG charts are successfully written to disk.
T-15	test_15_end_to_end_execution	run_full_pipeline()	Executes full pipeline and confirms complete evaluation dictionary output.